

Дизайн и анализ на алгоритми упражнение 12

Станислав Игов (Стъката)

2025/2026

Увод в динамичното програмиране. Няколко примера. Дефиниции. Задача за рестото

Част 1: Въвеждащи задачи

Задача 1: Числа на Фибоначи

Условие: Дадено е естествено число n . Да се намери n -тото число на Фибоначи, където $F(0) = 0$, $F(1) = 1$ и $F(n) = F(n-1) + F(n-2)$ за $n \geq 2$.

Рекурсивна декомпозиция

Дефиницията на числата на Фибоначи сама по себе си е рекурсивна — всяко число е сума от предходните две. Тоест задачата за пресмятане на $F(n)$ се свежда до две по-малки подзадачи: $F(n-1)$ и $F(n-2)$. Базовите случаи са $F(0) = 0$ и $F(1) = 1$.

```
Fib(n):  
@1  if n = 0: return 0  
@2  if n = 1: return 1  
@3  return Fib(n - 1) + Fib(n - 2)
```

Проблем. Наивната рекурсия преизчислява едни и същи подзадачи многократно. Например при пресмятане на $F(5)$ стойността $F(2)$ се изчислява 3 пъти, а $F(3)$ — 2 пъти. Времовата сложност е $O(2^n)$.

Прилагане на динамично програмиране (bottom-up)

Вместо рекурсия, пресмятаме стойностите последователно от най-малката подзадача нагоре.

```
Fib_DP(n):  
@1  dp[0] <- 0  
@2  dp[1] <- 1  
@3  for i from 2 to n:  
@4      dp[i] <- dp[i - 1] + dp[i - 2]  
@5  return dp[n]
```

Коректност

Тук коректността е почти очевидна, но нека бъдем внимателни, защото при по-сложните задачи ще разсъждаваме по същия начин.

Защо рекурентната връзка е вярна? Защото тя е дефиницията на числата на Фибоначи. Не правим нищо повече от това да кажем „ $F(n)$ е сумата на $F(n-1)$ и $F(n-2)$ “ — точно това пише в условието. Базовите случаи $F(0) = 0$ и $F(1) = 1$ също са дадени от дефиницията.

Защо bottom-up подходът работи? Представете си, че запълваме таблица отляво надясно: $dp[0], dp[1], dp[2], dp[3], \dots$. Когато стигнем до $dp[i]$, стойностите $dp[i-1]$ и $dp[i-2]$ вече са на местата си в таблицата — пресметнали

сме ги на предходни стъпки. Следователно $dp[i] = dp[i - 1] + dp[i - 2]$ ще бъде пресметнато от коректни стойности и самото то ще бъде коректно. Тази верига от разсъждения (ако по-малките подзадачи са верни, то и по-голямата е вярна) е по същество индукция — и точно по този начин ще аргументираме коректността при всяка ДП задача.

Времева сложност

$O(n)$ по време, $O(n)$ по памет (може да се оптимизира до $O(1)$, тъй като ни трябва само последните две стойности).

Задача 2: Опашка за билети

Условие: На опашка за билети стоят n души. Всеки може да си купи билет самостоятелно за време $t[i]$, или двама последователни хора могат да купят заедно (комбиниран билет) за време $p[i]$ (времето за комбинирана покупка на лицата i и $i + 1$). Да се намери минималното общо време за обслужване на всички хора.

Рекурсивна декомпозиция

Нека $dp[i]$ е минималното време за обслужване на първите i души. За i -тия човек има два варианта:

1. Той си купува билет сам — тогава минималното време е $dp[i - 1] + t[i]$.
2. Той купува заедно с $(i - 1)$ -вия — тогава минималното време е $dp[i - 2] + p[i - 1]$.

Вземаме минимума от двата варианта.

Базови случаи: $dp[0] = 0$ (няма хора — няма време), $dp[1] = t[1]$ (един човек — купува сам).

```
Tickets(n, t[1..n], p[1..n-1]):
@1  dp[1..n] <- malloc(n)
@2  dp[0] <- 0
@3  dp[1] <- t[1]
@4  for i from 2 to n:
@5      option1 <- dp[i - 1] + t[i]
@6      option2 <- dp[i - 2] + p[i - 1]
@7      dp[i] <- min(option1, option2)
@8  return dp[n]
```

Коректност

Имаме i души на опашката и искаме минималното общо време. Задаваме си въпроса: какво се случва с последния човек (човек i)?

Има точно две възможности:

- **Вариант А:** Човек i си купува билет сам. Тогава той отнема $t[i]$ време, а преди него трябва да обслужим оптимално първите $i - 1$ души. Общото време е $dp[i - 1] + t[i]$.
- **Вариант Б:** Човек i купува билет заедно с човек $i - 1$ (комбиниран билет). Тогава двамата заедно отнемат $p[i - 1]$ време, а преди тях трябва да обслужим оптимално първите $i - 2$ души. Общото време е $dp[i - 2] + p[i - 1]$.

Тези два варианта са изчерпващи, защото човек i или си купува сам, или купува в двойка с човек $i - 1$ — по условие няма друга опция (тройки не се допускат, прескачане на хора също не). Вземаме минимума, защото искаме минимално общо време.

Подзадачите са независими: в случай А решението за първите $i - 1$ души по никакъв начин не зависи от факта, че човек i купува сам. Аналогично в случай Б: решението за първите $i - 2$ души не зависи от двойката $(i - 1, i)$. Именно тази независимост позволява подзадачите да бъдат решени оптимално поотделно.

Пример

Нека $n = 4$ с данни: $t = [5, 2, 4, 3]$, $p = [6, 4, 5]$ (където $p[1]$ е за двойката $(1, 2)$, $p[2]$ за $(2, 3)$, $p[3]$ за $(3, 4)$).

i	$dp[i]$	Обяснение
0	0	база — няма хора
1	5	човек 1 купува сам: $dp[0] + t[1] = 0 + 5 = 5$
2	6	$\min(dp[1] + t[2], dp[0] + p[1]) = \min(7, 6) = 6$
3	9	$\min(dp[2] + t[3], dp[1] + p[2]) = \min(10, 9) = 9$
4	11	$\min(dp[3] + t[4], dp[2] + p[3]) = \min(12, 11) = 11$

Отговор: $dp[4] = 11$. По-изгодно е лица 3 и 4 да купят заедно (за 5), а преди тях първите двама да се обслужат оптимално (за 6). Общо: $6 + 5 = 11$.

Времева сложност

$$Time \in O(n)$$

Задача 3: Боядисване на къщи

Условие: Дадени са n къщи, подредени в редица. Всяка къща трябва да бъде боядисана в един от 3 цвята: червено (R), зелено (G) или синьо (B). Цената за боядисване на къща i в цвят c е $cost[i][c]$. Две съседни къщи не могат да бъдат в един и същ цвят. Да се намери минималната обща цена за боядисване на всички къщи.

Рекурсивна декомпозиция

Нека $dp[i][c]$ е минималната цена за боядисване на първите i къщи, при условие че къща i е боядисана в цвят c (където $c \in \{R, G, B\}$).

За да боядисаме къща i в цвят c , предходната къща $(i - 1)$ трябва да е в някой от другите два цвята. Тогава:

$$\begin{aligned} dp[i][R] &= cost[i][R] + \min(dp[i-1][G], dp[i-1][B]) \\ dp[i][G] &= cost[i][G] + \min(dp[i-1][R], dp[i-1][B]) \\ dp[i][B] &= cost[i][B] + \min(dp[i-1][R], dp[i-1][G]) \end{aligned}$$

Базов случай: $dp[1][c] = cost[1][c]$ за всяко $c \in \{R, G, B\}$.

```
PaintHouses(n, cost[1..n][R,G,B]): // за улеснение можем да си мислим че R = 1, G = 2, B = 3
@1 dp[1..n][R,G,B] <- malloc(3*n)
@2 dp[1][R] <- cost[1][R]
@3 dp[1][G] <- cost[1][G]
@4 dp[1][B] <- cost[1][B]
@5 for i from 2 to n:
@6     dp[i][R] <- cost[i][R] + min(dp[i-1][G], dp[i-1][B])
@7     dp[i][G] <- cost[i][G] + min(dp[i-1][R], dp[i-1][B])
@8     dp[i][B] <- cost[i][B] + min(dp[i-1][R], dp[i-1][G])
@9 return min(dp[n][R], dp[n][G], dp[n][B])
```

Коректност

Тази задача е малко по-сложна от предходните, защото състоянието вече не е само „колко къщи сме боядисали“, а „колко къщи сме боядисали и в какъв цвят е последната“. Защо трябва да помним цвета?

Представете си, че сте боядисали първите 4 къщи и сега решавате какъв цвят да дадете на 5-та. Единственото ограничение е, че 5-та не може да е в същия цвят като 4-та. Но за да проверите това, трябва да знаете

какъв е цветът на 4-та. Цветовете на къщи 1, 2 и 3 са напълно без значение за решението за къща 5 — те влияят само чрез факта, че вече са определили оптималната цена до къща 4.

Именно затова дефинираме $dp[i][c]$ — минималната цена за първите i къщи, като i -тата е в цвят c . Когато пресмятаме $dp[i][R]$ (цената за „къща i е червена”), знаем, че къща $i - 1$ трябва да е зелена или синя. Не ни интересува коя от двете — вземаме по-евтината:

$$dp[i][R] = cost[i][R] + \min(dp[i-1][G], dp[i-1][B])$$

Тук $\min(dp[i-1][G], dp[i-1][B])$ е минималната цена за боядисване на първите $i - 1$ къщи при условие, че $(i - 1)$ -вата не е червена. Тъй като $(i - 1)$ -вата къща може да е или зелена, или синя (и двата варианта са допустими), вземаме минимума.

Подзадачата за първите $i - 1$ къщи е напълно самостоятелна, стига да знаем цвета на $(i - 1)$ -вата къща. Как сме боядисали къщи $1, 2, \dots, i - 2$ не зависи от цвета на къща i — зависи само от цвета на къща $i - 1$, и точно тази информация пазим в състоянието.

Финалният отговор е $\min(dp[n][R], dp[n][G], dp[n][B])$, защото за последната къща няма ограничение „на-дясно” и всеки цвят е допустим.

Пример

Нека $n = 3$ с цени:

	R	G	B
Къща 1	17	2	17
Къща 2	16	16	5
Къща 3	14	3	19

Базов случай ($i = 1$): $dp[1][R] = 17$, $dp[1][G] = 2$, $dp[1][B] = 17$.
 $i = 2$:

$$\begin{aligned} dp[2][R] &= 16 + \min(2, 17) = 18 \\ dp[2][G] &= 16 + \min(17, 17) = 33 \\ dp[2][B] &= 5 + \min(17, 2) = 7 \end{aligned}$$

$i = 3$:

$$\begin{aligned} dp[3][R] &= 14 + \min(33, 7) = 21 \\ dp[3][G] &= 3 + \min(18, 7) = 10 \\ dp[3][B] &= 19 + \min(18, 33) = 37 \end{aligned}$$

Отговор: $\min(21, 10, 37) = 10$. Оптималното боядисване е: къща 1 — зелена (2), къща 2 — синя (5), къща 3 — зелена (3). Общо: $2 + 5 + 3 = 10$.

Забележете, че къщи 1 и 3 са и двете зелени, което е позволено — ограничението е само за съседни къщи.

Времева сложност

$O(n)$ по време и $O(n)$ по памет (или $O(1)$ с оптимизация, пазейки само предходния ред).

Част 2: Какво е динамично програмиране?

Определение

Динамичното програмиране (ДП) е метод за решаване на оптимизационни задачи (или задачи за броене), при който разбиваме задачата на по-малки, припокриващи се подзадачи, решаваме всяка подзадача точно веднъж и комбинираме резултатите.

Кога е приложимо?

Динамичното програмиране е приложимо, когато задачата притежава две ключови свойства:

1. Оптимална подструктура. Оптималното решение на задачата може да бъде конструирано от оптималните решения на нейните подзадачи. С други думи, ако вземем оптимално решение и погледнем как то решава подзадачите, то решава и тях оптимално.

В горните примери:

- *Фибоначи*: $F(n) = F(n-1) + F(n-2)$ — стойността зависи от стойностите на подзадачите.
- *Билети*: Оптималното обслужване на i човека включва оптимално обслужване на първите $i-1$ или $i-2$.
- *Къщи*: Оптималното боядисване на i къщи с фиксиран цвят за i -тата включва оптимално боядисване на първите $i-1$.

2. Припокриващи се подзадачи. Различните разклонения на рекурсията водят до едни и същи подзадачи. Ако всяка подзадача се среща само веднъж, нямаме нужда от мемоизация и ДП е излишно (разделяй и владей е достатъчно).

В горните примери:

- *Фибоначи*: $F(3)$ се използва и при пресмятане на $F(5)$, и при $F(4)$.
- *Билети*: $dp[i-2]$ е необходимо за $dp[i]$, но е вече пресметнато за $dp[i-1]$.
- *Къщи*: $dp[i-1][c]$ се използва при пресмятане на $dp[i][c']$ за $c' \neq c$.

Схема за решаване на задачи с ДП

1. **Дефинирай подзадачите.** Опиши какво точно пресмята $dp[i]$ (или $dp[i][j], \dots$).
2. **Намери рекурентната връзка.** Как $dp[i]$ зависи от по-малки подзадачи?
3. **Определи базовите случаи.** Кои са най-малките подзадачи, чийто отговор знаем директно?
4. **Определи реда на пресмятане.** В какъв ред да пресмятаме, за да са готови подзадачите, от които зависим?
5. **Определи крайния отговор.** Коя клетка от таблицата (или коя комбинация от клетки) дава окончателния отговор?

Два подхода

- **Bottom-up (табулация).** Запълваме таблица от базовите случаи нагоре. Това е итеративният подход, който видяхме в горните примери.
- **Top-down (мемоизация).** Пишем рекурсивна функция, но запомняме (кешираме) резултата от всяка подзадача, за да не я пресмятаме повторно.

И двата подхода имат еднаква времева сложност. Bottom-up е по-лесен за анализ на паметта, а top-down е по-интуитивен, защото следва естествената рекурсивна декомпозиция.

Част 3: Задача за връщане на ресто

Задача 4: Coin Change (Ресто с произволна деноминация)

Условие: Дадени са k вида монети с деноминации $d[1], d[2], \dots, d[k]$ и целева сума S . Всяка монета може да се използва неограничен брой пъти. Да се намери минималният брой монети, необходими за съставяне на сумата S . Ако е невъзможно, да се върне -1 .

Стъпка 1: Дефиниция на подзадачите

Нека $dp[s]$ е минималният брой монети, необходими за съставяне на сума s , за $0 \leq s \leq S$.

Стъпка 2: Рекурсивна декомпозиция

Разглеждаме последната монета, добавена към сумата. Ако тази монета е $d[j]$, то преди нея вече сме съставили сума $s - d[j]$. Следователно:

$$dp[s] = \min_{\substack{1 \leq j \leq k \\ d[j] \leq s}} (dp[s - d[j]] + 1)$$

Тоест за всяка сума s изпробваме всяка възможна монета като последна и вземаме варианта с минимален брой.

Стъпка 3: Базов случай

$dp[0] = 0$ (за сума 0 не са нужни монети).

Стъпка 4: Ред на пресмятане

Пресмятаме $dp[1], dp[2], \dots, dp[S]$ в нарастващ ред. Когато пресмятаме $dp[s]$, всички $dp[s - d[j]]$ вече са пресметнати, защото $s - d[j] < s$.

Стъпка 5: Краен отговор

Отговорът е $dp[S]$. Ако $dp[S] = \infty$ (маркирано като недостижимо), връщаме -1 .

Псевдокод

```
CoinChange(S, d[1..k]):
@1  dp[1..S] <- malloc(S)
@2  dp[0] <- 0
@3  for s <- 1 to S:
@4      dp[s] <- infinity
@5      for j from 1 to k:
@6          if d[j] <= s and dp[s - d[j]] + 1 < dp[s]:
@7              dp[s] <- dp[s - d[j]] + 1
@8  if dp[S] = infinity:
@9      return -1
@10 else:
@11     return dp[S]
```

Коректност

Защо рекурентната връзка е коректна?

Нека помислим какво означава „да съставим сума s с минимален брой монети“. Имаме някакъв набор от монети, които сумарно дават s . Задаваме ключовия въпрос: *коя е последната монета, която слагаме?*

Тя е някоя от наличните деноминации — да кажем $d[j]$. След като я сложим, остава да съставим сума $s - d[j]$ от останалите монети. И тук идва критичното наблюдение: тези останали монети *трябва* да са оптимално решение за сумата $s - d[j]$.

Защо? Да допуснем обратното — че сме съставили $s - d[j]$ с t' монети, но съществува начин да го направим с $t'' < t'$ монети. Тогава бихме заменили нашите t' монети с t'' и щяхме да получим по-добро решение за s (с $t'' + 1$ вместо $t' + 1$ монети). Но ние казахме, че имаме оптимално решение за s — противоречие!

Следователно, ако знаем коя е последната монета, оптималното решение за s е $1 +$ оптималното решение за $s - d[j]$. Тъй като *не знаем* коя е последната монета, опитваме всички възможни деноминации $d[1], d[2], \dots, d[k]$ (стига $d[j] \leq s$) и вземаме минимума. Гарантирано сме разгледали и правилния вариант, защото сме обхванали всички.

Защо грийди подход не работи?

Може да изглежда, че е достатъчно винаги да вземаме най-голямата монета, която се побира. Но това не е вярно при произволни деноминации! Например с монети $\{1, 3, 4\}$ и сума 6: грийди подходът взема $4 + 1 + 1 = 3$ монети, но оптималното решение е $3 + 3 = 2$ монети. Именно затова ДП е нужно — то изпробва всички варианти систематично.

Пример

Монети: $d = \{1, 3, 4\}$, Сума $S = 6$.

Пресмятаме $dp[s]$ за $s = 0, 1, \dots, 6$. За всяко s проверяваме всяка монета $d[j] \in \{1, 3, 4\}$, стига $d[j] \leq s$.

s	Проверки	$dp[s]$	Обяснение
0	—	0	база
1	$d=1: dp[0] + 1 = 1$	1	единствена възможност
2	$d=1: dp[1] + 1 = 2$	2	две монети от 1
3	$d=1: 3; d=3: dp[0] + 1 = 1$	1	една монета от 3
4	$d=1: 2; d=3: 2; d=4: dp[0] + 1 = 1$	1	една монета от 4
5	$d=1: 2; d=3: 3; d=4: 2$	2	$4 + 1$ или $1 + 4$
6	$d=1: 3; d=3: dp[3] + 1 = 2; d=4: 3$	2	$3 + 3$ — две монети!

Отговор: $dp[6] = 2$ (монети $3 + 3$).

Забележете какво се случи при $s = 6$: монетата от 1 дава $dp[5] + 1 = 3$, монетата от 4 дава $dp[2] + 1 = 3$, но монетата от 3 дава $dp[3] + 1 = 1 + 1 = 2$ — най-добрият вариант. Грийди подходът (вземи най-голямата монета първо) щеше да избере 4, после 1, после 1, и щеше да даде 3 монети вместо 2.

Времева сложност

$$Time \in O(S \cdot k)$$

По памет: $O(S)$.

Обобщение

Задача	Подзадача	Рекурентна връзка	Сложност
Фибоначи	$dp[i] = i$ -тото число	$dp[i] = dp[i-1] + dp[i-2]$	$O(n)$
Билети	$dp[i] =$ мин. време за i души	$dp[i] = \min(dp[i-1] + t[i], dp[i-2] + p[i-1])$	$O(n)$
Къщи	$dp[i][c] =$ мин. цена, i -та в c	$dp[i][c] = cost[i][c] + \min_{c' \neq c} dp[i-1][c']$	$O(n)$
Ресто	$dp[s] =$ мин. монети за сума s	$dp[s] = \min_j (dp[s - d[j]] + 1)$	$O(S \cdot k)$

Ключовият урок: при всяка задача дефинираме подзадача, намираме как оптималното решение на по-голямата задача зависи от оптималните решения на по-малките, и пресмятаме систематично (отдолу нагоре или отгоре надолу с мемоизация).